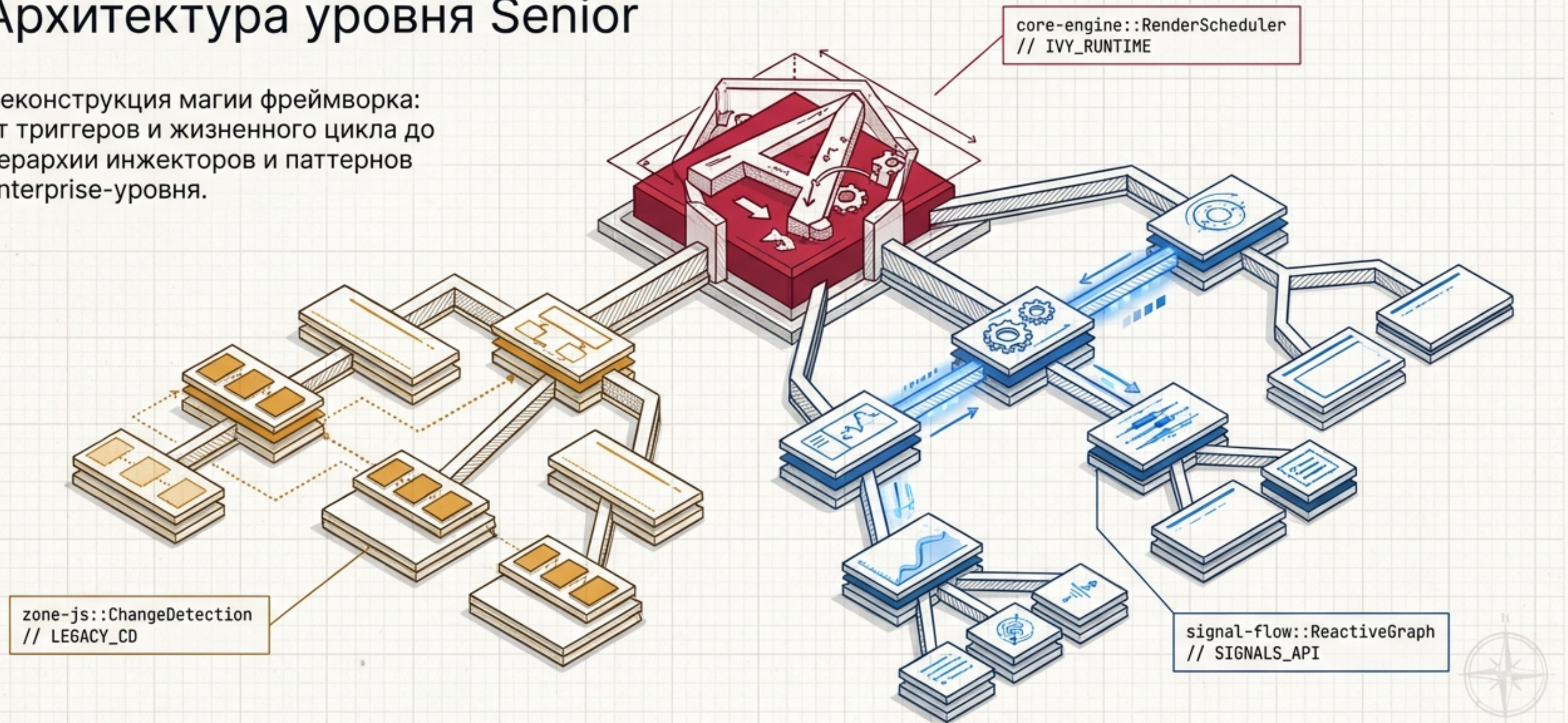


Внутреннее устройство Angular: Архитектура уровня Senior

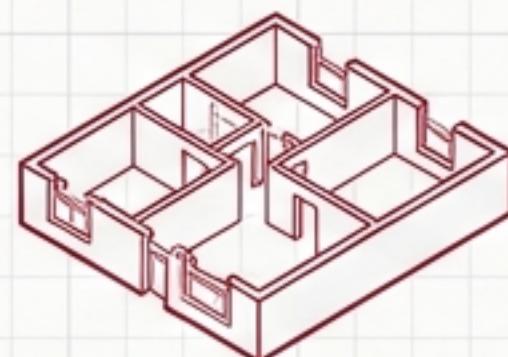
Деконструкция магии фреймворка:
от триггеров и жизненного цикла до
иерархии инжекторов и паттернов
Enterprise-уровня.



Разделение слоёв: Триггер против Стратегии



Кто позвонил
в дверь

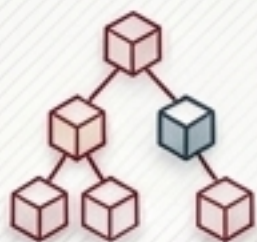


Как проверяют
дом внутри



Главная ошибка мышления — считать, что OnPush заменяет Zone.js, или что Zoneless отключает OnPush. Это два независимых уровня системы, которые всегда работают в тандеме.

Жизненный цикл и синхронизация Change Detection



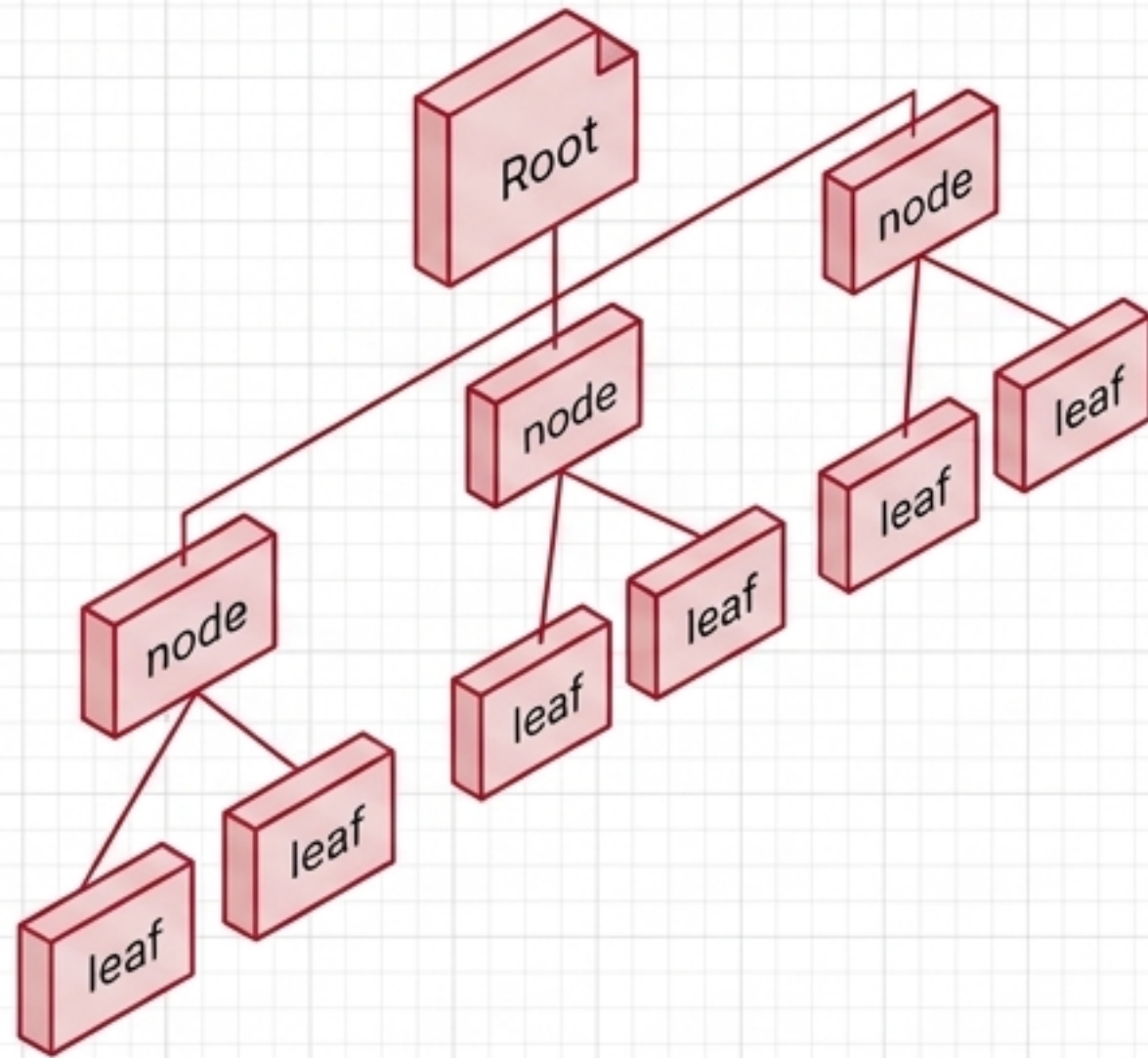
Angular проходит по дереву компонентов, сверяет биндинги и только после этого обновляет изменившийся DOM.



Стратегии проверки: Глобально vs Точечно

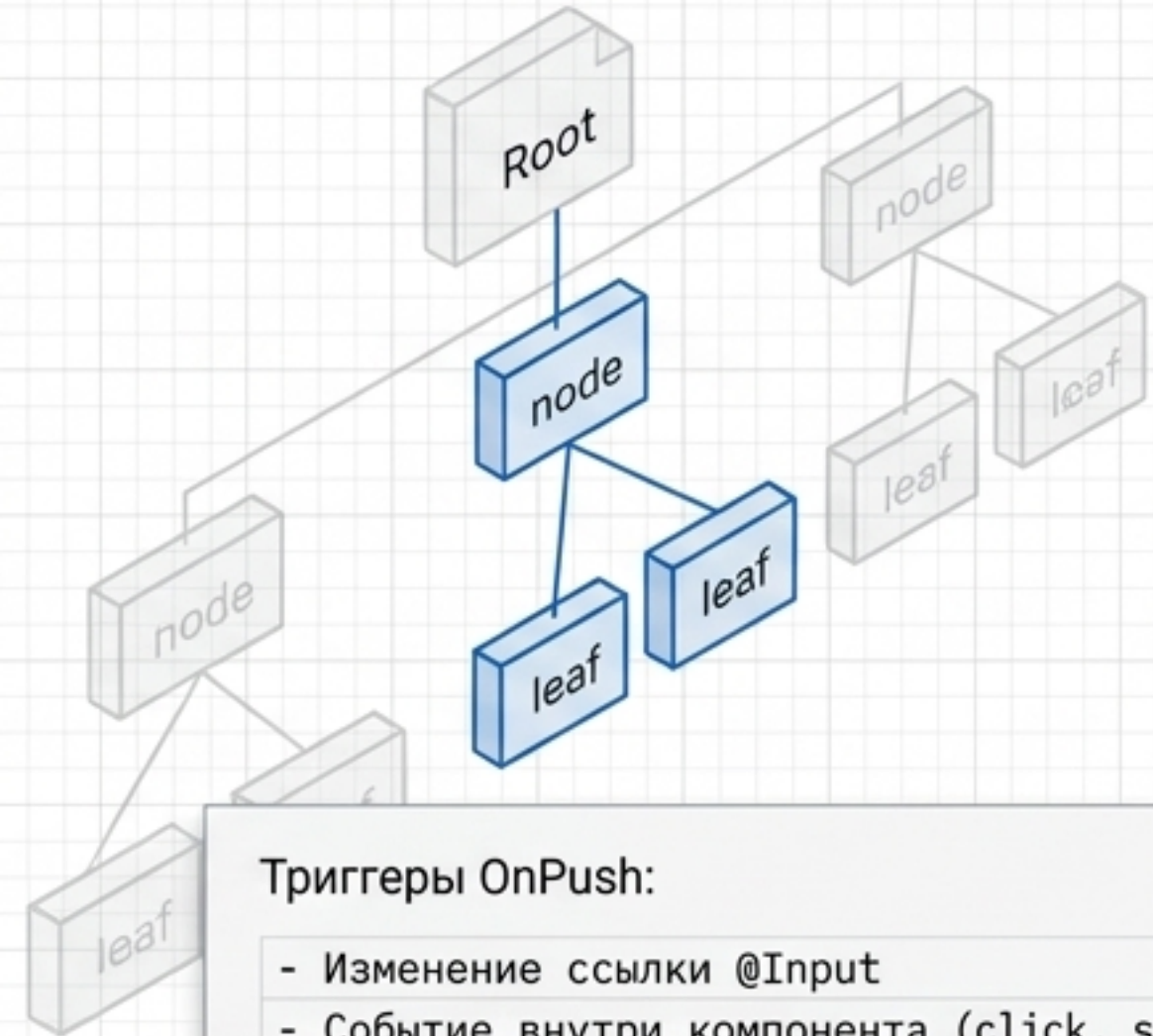
Default Strategy

Проверяется всё дерево, независимо от источника изменений. Сравнение по значению (Value).



OnPush Strategy

Проверка только по явному триггеру. Сравнение по ссылке (Reference).



Триггеры OnPush:

- Изменение ссылки @Input
- Событие внутри компонента (click, submit)
- Срабатывание async pipe
- Изменение Signal
- Ручной вызов markForCheck()

Управление вручную: ChangeDetectorRef

Отложенная проверка

```
this.cdr.markForCheck();
```



Говорит Angular: «Проверь этот компонент позже» (при следующем цикле CD). Не запускает синхронную проверку. Идеально для батчинга данных.

Синхронная проверка

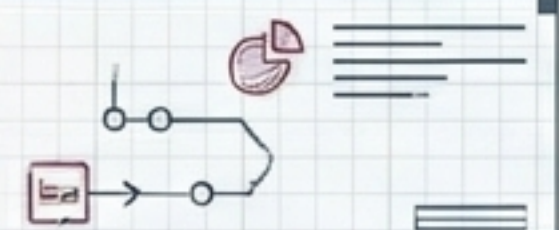
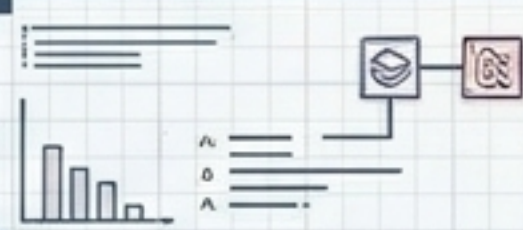
```
this.cdr.detectChanges();
```



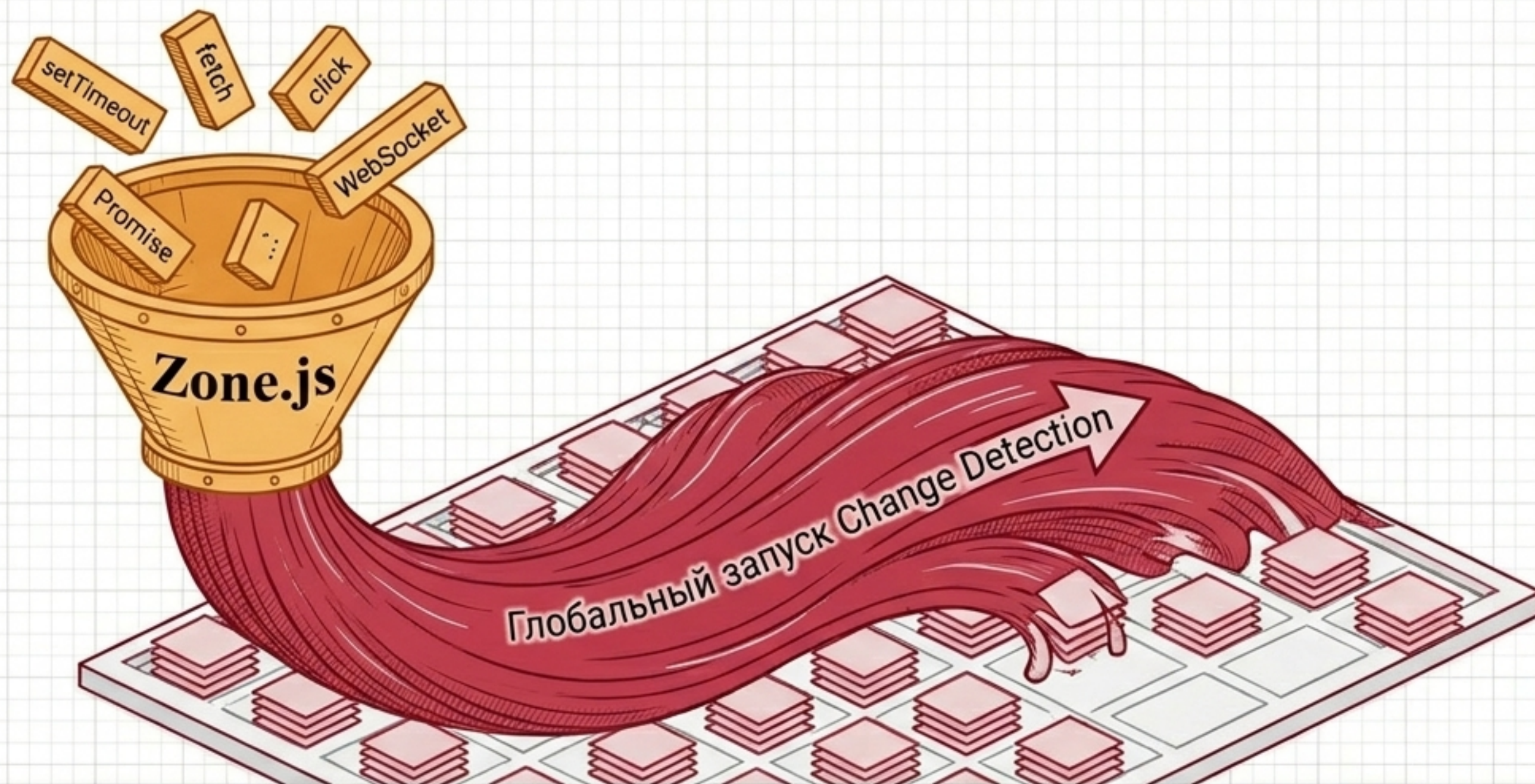
Синхронно заставляет Angular немедленно проверить компонент и его детей. Блокирует поток.

Enterprise Кейс: Обработка тяжелых потоков WebSocket.

Использование `markForCheck` накапливает изменения, предотвращая фризы UI, характерные для постоянных `detectChanges`.



Эпоха Zone.js: Глобальный перехват API



Проблема производительности:

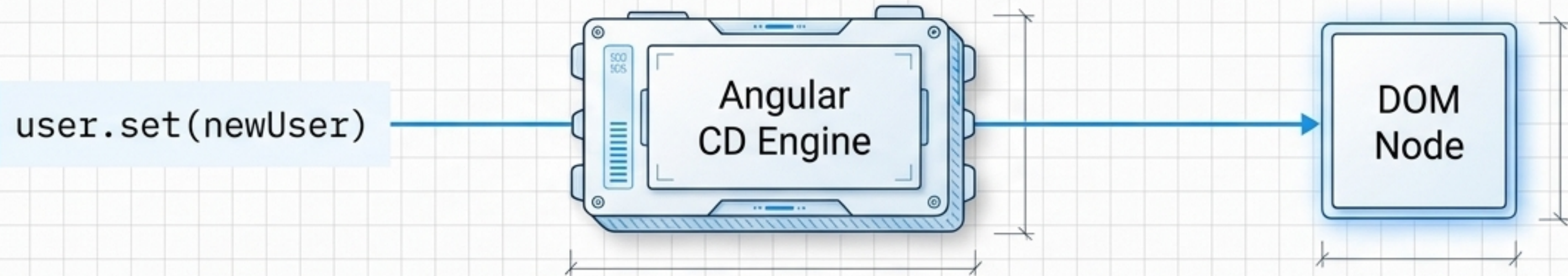
В тяжелых дашбордах частые WebSocket-сообщения вызывают лавину CD.

Escape Hatch (Спасательный люк):

```
this.ngZone.runOutsideAngular(() => { ... })
```

Выполнение кода без триггера глобального CD.

Новый мир: Zoneless Angular и Сигналы



Отказ от глобального слушателя в пользу явных уведомлений:
`provideZonelessChangeDetection()`

Crucial Senior Insight

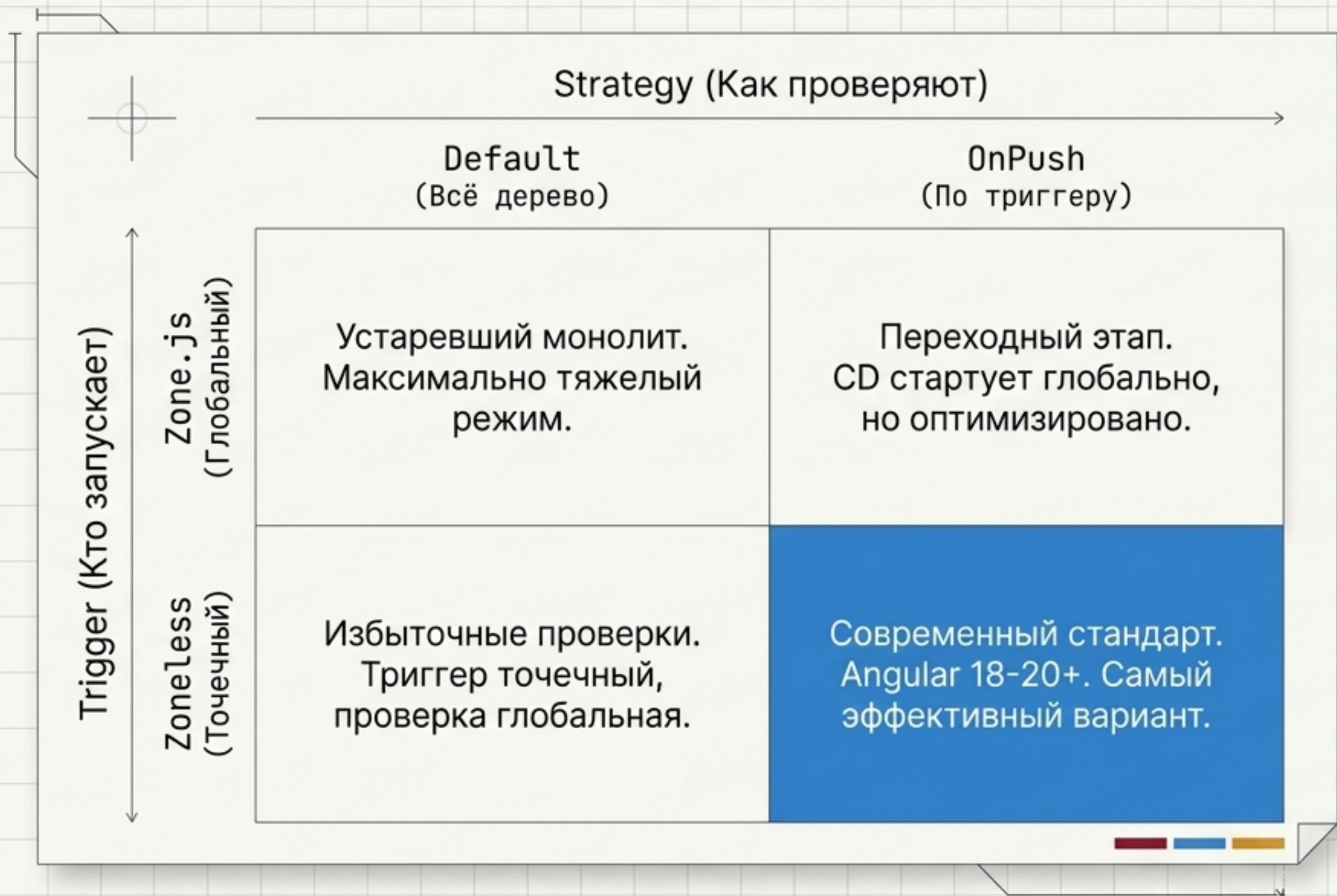


Миф: Сигналы обновляют DOM напрямую в обход CD.



Реальность: Signal сообщает Angular: «Данные изменились». Angular запускает CD (более предсказуемо) и только затем обновляет DOM.

Эволюция архитектуры: Матрица рендеринга



Эволюция Dependency Injection: функция inject()

Legacy: Class-bound

```
constructor(private http: HttpClient) {}
```

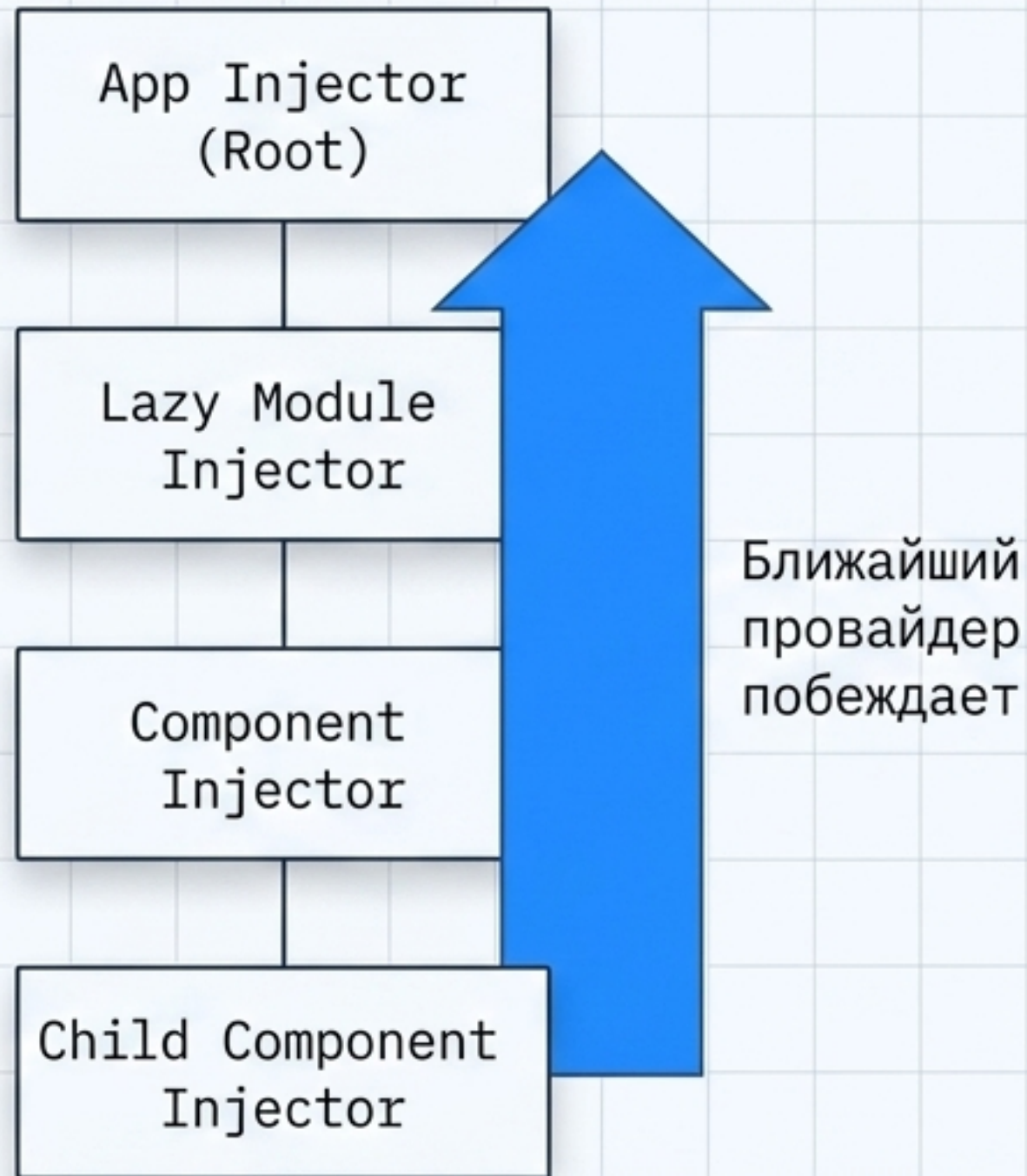
Modern: Functional API

```
private http = inject(HttpClient);
```

Почему inject() архитектурно превосходит конструктор:

1. Работает вне классов (Functional Angular).
2. Идеально для Guards (`export const authGuard = () => inject(AuthService)`).
3. Упрощает создание Interceptors, Standalone functions, Stores и Signals.

Дерево инжекторов: Алгоритм поиска зависимостей

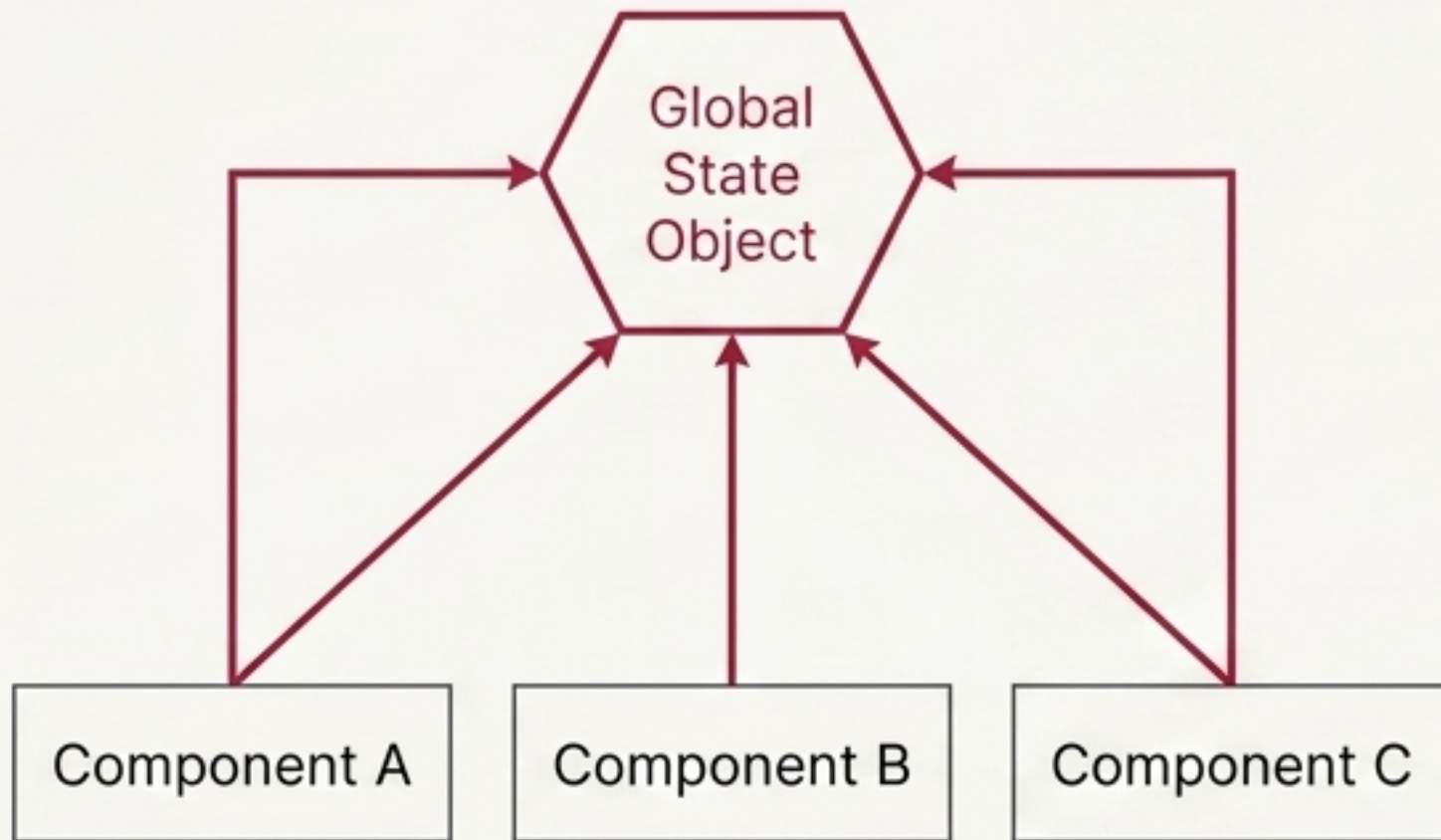


Алгоритм резолюции:

- `Angular` ищет провайдер в текущем компоненте.
- Если не находит – поднимается вверх по дереву вплоть до `root`.
- Если достигает вершины и провайдера нет – выбрасывает `NullInjectorError`.

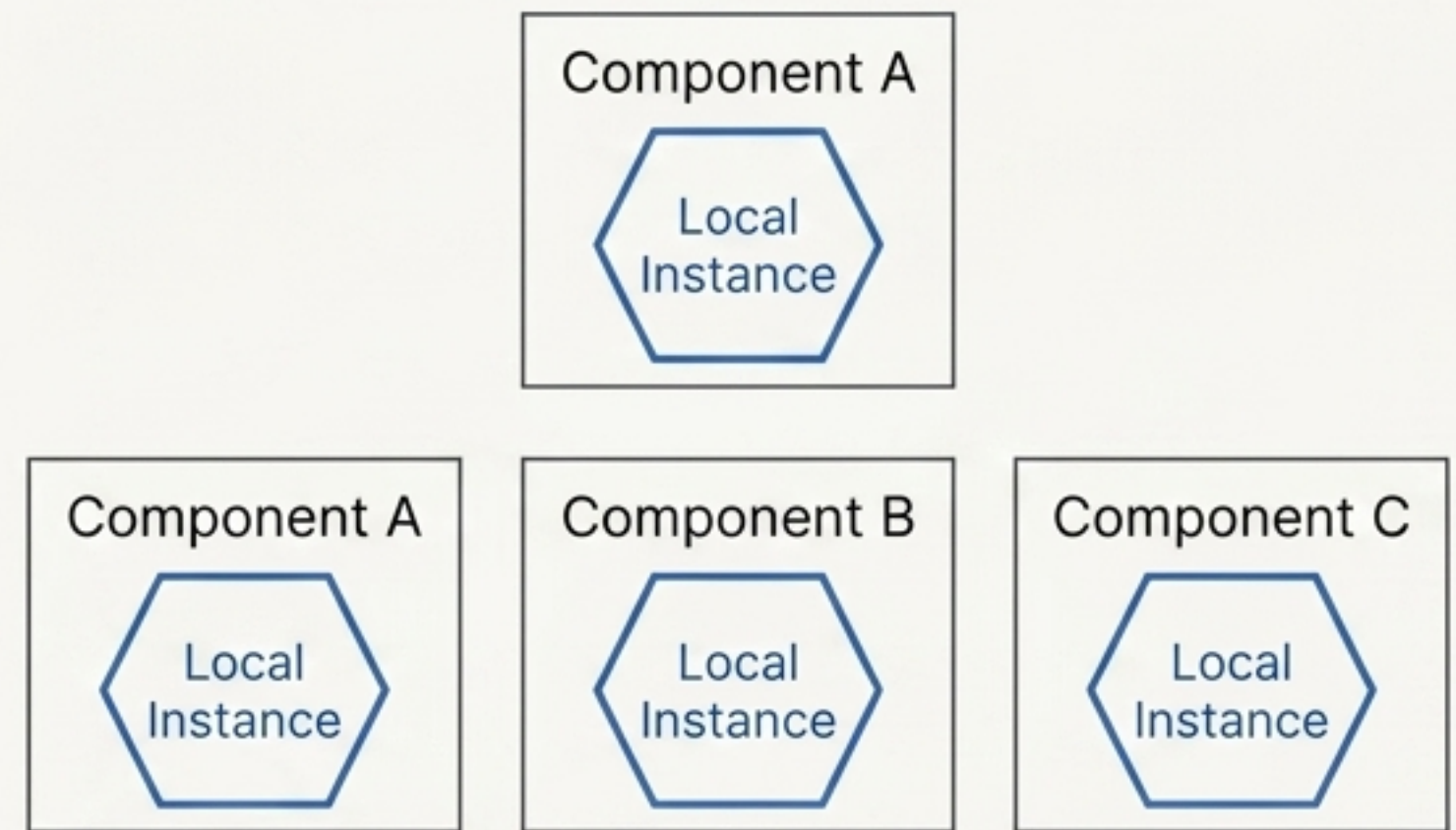
Область видимости: Singleton vs Локальный стейт

`providedIn: 'root'`



Один общий экземпляр (Singleton).
Автоматически регистрируется и
поддерживает tree-shaking.

`providers: [...]` в компоненте

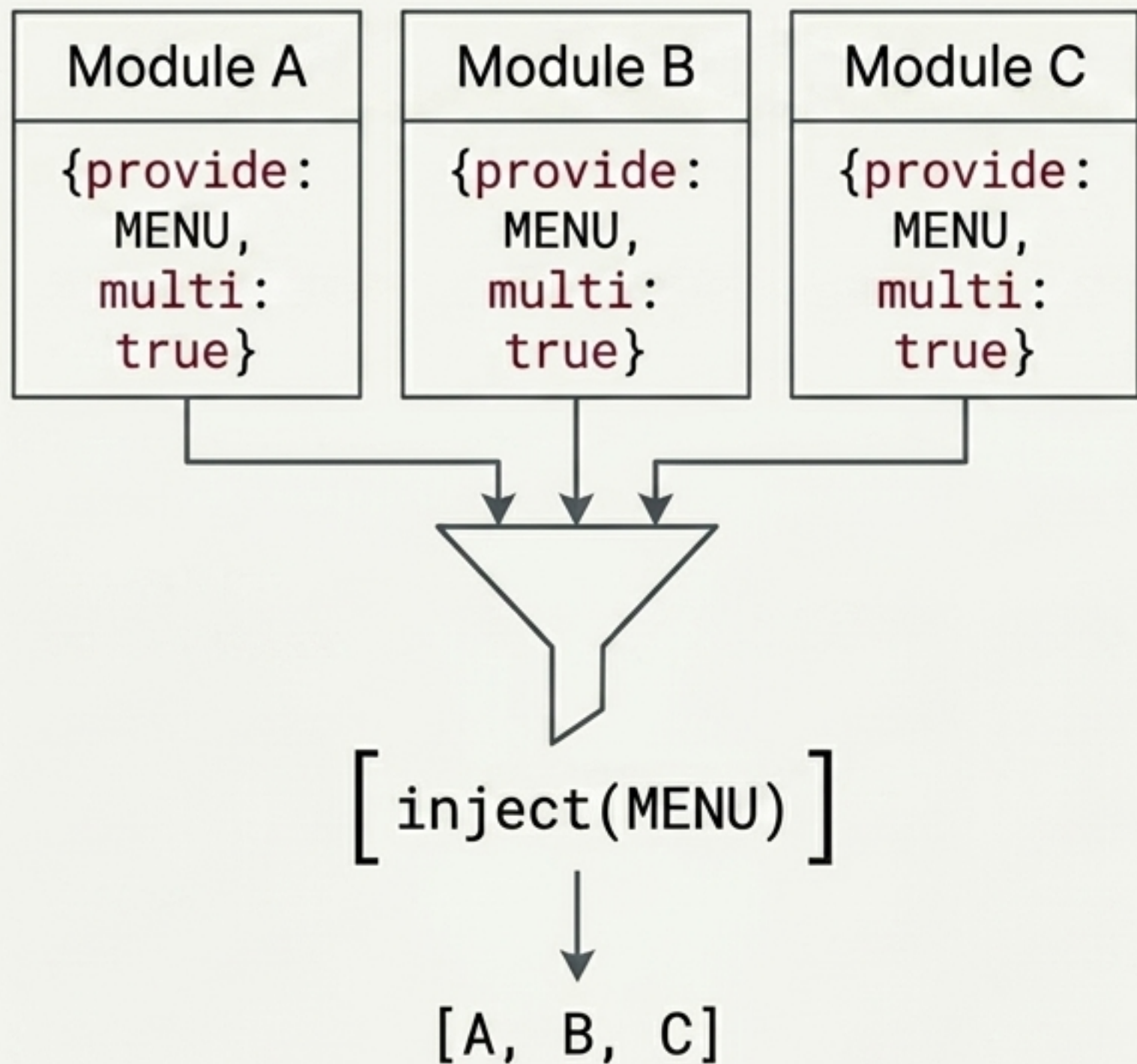


Новый изолированный экземпляр для
каждого компонента и его детей.
Идеально для локального стейта.

Продвинутые провайдеры: Абстракция и Алиасы

Паттерн	Инструмент	Код	Use Case
Отсутствие интерфейсов в runtime	InjectionToken	<pre>new InjectionToken<ApiConfig>('API_CONFIG')</pre>	Внедрение не-классовых зависимостей (конфиги, environment settings).
Подмена реализации	useClass	<pre>{ provide: Logger, useClass: FileLogger }</pre>	Dev vs Prod, Mock-сервисы в тестах без изменения логики компонентов.
Совместимость API	useExisting	<pre>{ provide: OldLogger, useExisting: NewLogger }</pre>	Алиасы, постепенный рефакторинг легаси-кода.

Продвинутые провайдеры: Фабрики и Плагины



Enterprise Plugins (multi: true)

Сборка массива из разных независимых провайдеров.

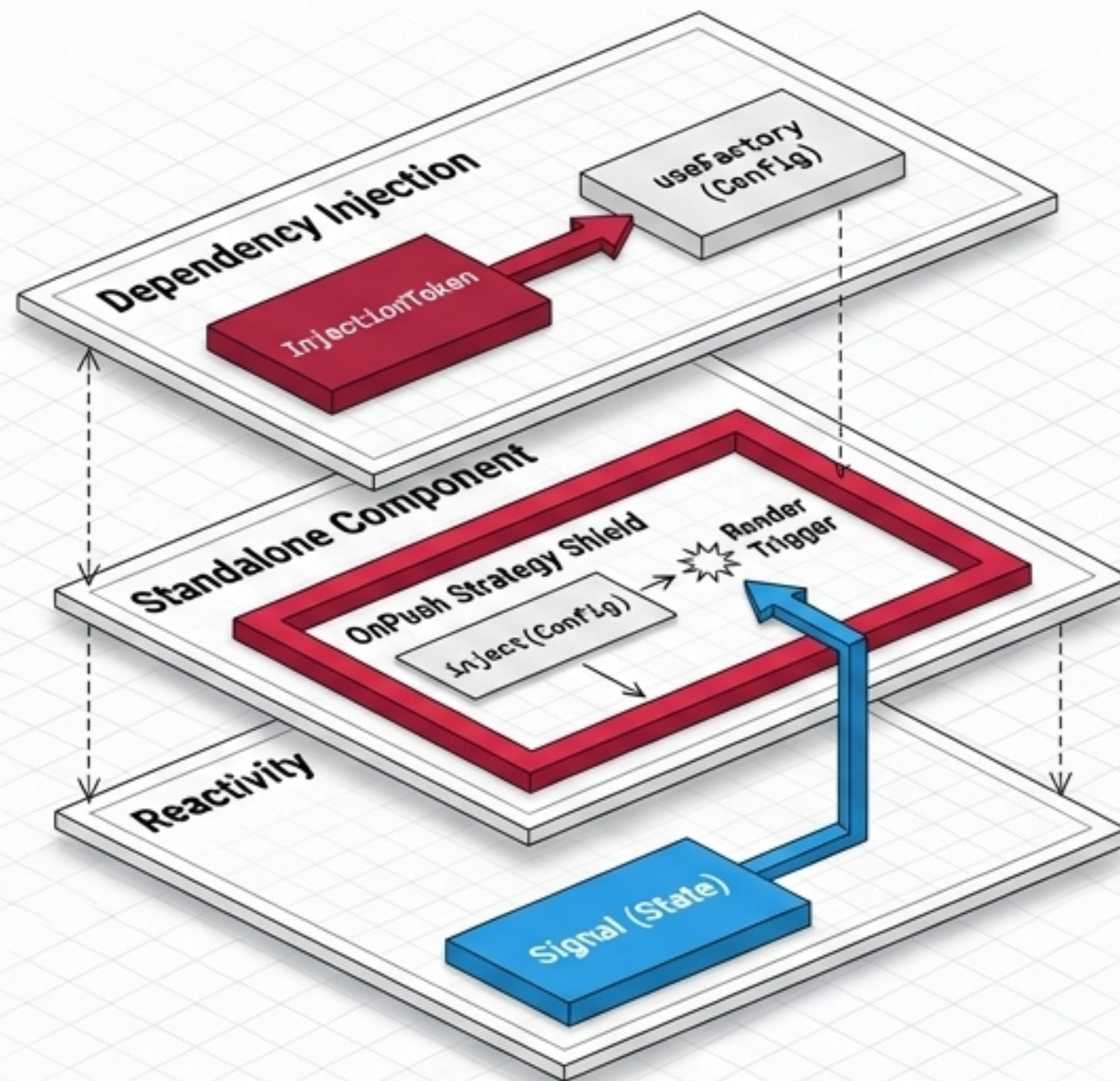
- Кейсы: HTTP Interceptors цепочки, системы меню, middleware, хуки событий.

Dynamic Creation (useFactory)

Создание зависимостей динамически.

```
useFactory: () => env.prod ?  
'https://prod' : 'http://local'
```


Синтез: Архитектура современного Feature-модуля



Настоящая Senior-архитектура — это конфигурация через DI, реактивный стейт через Signals и точечный рендеринг через OnPush, полностью независимый от Zone.js.

Итоги и Check-list архитектора

Change Detection

- ✗ Использование `detectChanges` вместо `markForCheck` для батчинга данных WebSocket.
- ✗ Мутация объектов (`value`) при включенной стратегии `OnPush` (ожидается `reference`).

Реактивность

- ✗ Вера в то, что `Signals` обновляют DOM в обход механизма Change Detection.
- ✗ Забытый `runOutsideAngular` для тяжелых операций в приложениях с `Zone.js`.

DI Система

- ✗ Потеря `providedIn: 'root'` и возникновение `NullInjectorError`.
- ✗ Хардкод конфигураций в классах вместо использования `InjectionToken` и `multi: true`.